

Unit 5



**Ganpat
University**

॥ विद्यया समाजोत्कर्षः ॥

Department of
Computer Science



B.Sc. Hons. (CA&IT) Sem-I
U11A1IP1-INTRODUCTION TO
PROGRAMMING-I
UNIT-5
Array and Strings

Prepared By:

Akash Chaudhary

Department of Computer Science Ganpat University

1.1 Introduction to Array

What is an Array?

An array is a **group of similar types of data stored under one variable name**, using an index to access individual elements.

→ Declaring a 1D Array

Syntax:

```
data_type array_name[size];
```

Example:

```
int marks[5];
```

→ Accessing Array Elements

```
#include <stdio.h>
void main() {
    int marks[5] = {10, 20, 30, 40, 50};

    printf("First value = %d\n", marks[0]);
    printf("Third value = %d\n", marks[2]);
}
```

Output:

```
First value = 10
Third value = 30
```

In C, **index starts from 0**. So **marks[0]** is the **first element**, not marks[1].

1.2 Concept of Dimensions in Arrays

	Syntax	Example
1D (One-Dimensional)	data_type array_name[size];	<pre>int nums[4] = {5, 10, 15, 20}; Indexes are: nums[0] = 5 nums[1] = 10</pre>

		<pre>nums[2] = 15 nums[3] = 20</pre>
2D (Two-Dimensional)	<pre>data_type array_name[rows][columns];</pre>	<pre>int mat[2][3] = { {1, 2, 3}, {4, 5, 6} };</pre> Accessing elements: <pre>mat[0][1] = 2 mat[1][2] = 6</pre>
3D (Three-Dimensional)	<pre>data_type array_name[depth][rows][columns];</pre>	<pre>int cube[2][2][2] = { { {1, 2}, {3, 4} }, { {5, 6}, {7, 8} } };</pre> Accessing: <pre>cube[1][0][1] = 6</pre>

(A **multidimensional array** means an array with more than one index (row and column, or more).

1.3 Initialization Values in an Array

Full Initialization: <pre>int arr[5] = {1, 2, 3, 4, 5};</pre>	Partial Initialization: <pre>int arr[5] = {1, 2};</pre> Remaining values will be automatically 0: <pre>arr[0] = 1 arr[1] = 2 arr[2] = 0 arr[3] = 0 arr[4] = 0</pre>
---	--

1.4 Overflow and Underflow

Array Overflow:

Happens when we try to access an index outside the array size.

```
int arr[3] = {10, 20, 30};  
printf("%d", arr[3]); // Invalid! Index out of bounds
```

Valid indexes are 0, 1, 2. Accessing `arr[3]` is overflow.

Array Underflow:

Happens when we access a negative index.

```
int arr[3] = {10, 20, 30};  
printf("%d", arr[-1]); // Invalid! Negative index
```

This is called underflow.

Accessing `invalid indexes` can cause errors or crashes.

★ Loop Example to Print 2D Array:

```
#include <stdio.h>  
void main() {  
    int mat[2][3] = {  
        {1, 2, 3},  
        {4, 5, 6} };  
  
    int i, j;  
    for(i = 0; i < 2; i++) {  
        for(j = 0; j < 3; j++) {  
            printf("%d ", mat[i][j]); }  
        printf("\n");  
    }  
}
```

Output:

```
1 2 3  
4 5 6
```

2.1 Introduction to Strings

- A string is a **group of characters** (letters, digits, symbols).

Example:

```
char name[] = "Amit";
```

This is same as:

```
char name[] = {'A', 'm', 'i', 't', '\0'};
```

2.2 Reading String from Terminal

- **Using scanf (won't accept spaces)**

```
#include <stdio.h>
#include <conio.h>
void main() {
    char name[20];
    clrscr();
    printf("Enter your name: ");
    scanf("%s", name);
    printf("%s", name);
    getch();
}
```

- **Use gets() to read full line (in Turbo C):**

```
#include <stdio.h>
#include <conio.h>

void main() {
    char sentence[50];
    clrscr();
    printf("Enter a sentence: ");
    gets(sentence); // reads full line with spaces
    printf("%s", sentence);
    getch();
}
```

2.3 Writing Strings to the Screen

You can display your strings using these functions.

1) Using `printf()`

- The standard way to print formatted output.
- Example: `printf("Hello %s", name);`

2) Using `puts()`

- A simpler function that prints a string and automatically adds a new line.
- Example: `puts(name);`

Function	Description	Example
<code>getchar()</code>	Reads a single character from keyboard	<code>char c = getchar();</code>
<code>gets()</code>	Reads a string (line) from keyboard until newline	<code>char str[50]; gets(str);</code>
<code>putchar()</code>	Prints a single character on screen	<code>putchar('A');</code>
<code>puts()</code>	Prints a string followed by a newline	<code>puts("Hello");</code>

2.4 Arithmetic Operations on Characters

Characters are stored as ASCII values, so you can do arithmetic.

Example:

```
#include <stdio.h>

void main() {
    char ch = 'A';
    ch = ch + 1;
    printf("Next character: %c", ch); // Output: B
}
```

2.5 String Handling Functions

C provides a set of built-in functions in the `<string.h>` library to handle strings efficiently.

1) `strlen()`

- **Purpose:** Returns the length of a string (excluding `\0`).
- **Example:**

```
#include <stdio.h>
#include <conio.h>
#include <string.h>

void main() {
    char str[] = "Hello";
    clrscr();
    printf("Length of string: %d\n", strlen(str));
    getch();
}
```

Output:

Length of string: 5

2) `strcpy()`

- **Purpose:** Copies one string into another.
- **Example:**

```
#include <stdio.h>
#include <string.h>

void main() {
    char str1[20];
    char str2[] = "Hello";
    strcpy(str1, str2);
    printf("Copied string: %s\n", str1);
}
```

Output:

Copied string: Hello

3) **strcat()**

- **Purpose:** Concatenates (joins) two strings.
- **Example:**

```
#include <stdio.h>
#include <string.h>

void main() {
    char str1[20] = "Hello ";
    char str2[] = "World";
    strcat(str1, str2);
    printf("Concatenated string: %s\n", str1);
}
```

Output:

Concatenated string: Hello World

4) **strcmp()**

- **Purpose:** Compares two strings.
- **Returns:**
 - 0 → if strings are equal
 - <0 → if **str1 < str2**
 - >0 → if **str1 > str2**
- **Example:**

```
#include <stdio.h>
#include <string.h>

void main() {
    char str1[] = "Apple";
    char str2[] = "Banana";
    int result = strcmp(str1, str2);
    printf("Comparison result: %d\n", result);
}
```

Output:

Comparison result: -1 (or a negative number)

5) **strrev()** (Not standard in C, may work in some compilers)

- **Purpose:** Reverses a string.
- **Example:-**

```
#include <stdio.h>
#include <string.h>

void main() {
    char str[] = "Hello";
    strrev(str);
    printf("Reversed string: %s\n", str);
}
```

Output:

Reversed string: olleH

String Handling Functions (Summary Table)

Function	Purpose	Example Use
strcpy()	Copy string	strcpy(a, b);
strcmp()	Compare strings	strcmp(a, b);
strcat()	Join two strings	strcat(a, b);
strlen()	Find length of string	strlen(a);